

HomelabSentinel — A Complete Teaching Guide

Building a production agentic-AI system in Python, file by file

This guide teaches the entire HomelabSentinel project: what each file does, why it exists, how the pieces fit, and — most importantly — the protection mechanisms that let an LLM safely touch real infrastructure. It follows the project's own built-in curriculum: the `agent_v1` through `agent_v5` files solve the same task five times, each adding one production concern. Read them in order and you have learned how to build agents.

Part 0 — The mental model

HomelabSentinel is an SRE (Site Reliability Engineer) that lives in your homelab and that you talk to in plain English. You ask "are my backups OK?" or "restart Immich" over Telegram (or Alexa), and an LLM reasons about it: it picks tools, gathers live data from Proxmox / Home Assistant / Portainer, decides whether an action is needed, and — crucially — asks your permission before it changes anything.

The single most important idea in the whole project is this split between reading the world and changing the world:

READING the world (safe, free, frequent) ----- check_proxmox_status check_reachability check_backups check_presence_state search_docs	CHANGING the world (dangerous, gated, rare) ----- restart_lxc restart_docker_container call_ha_service (lights / heat) ^ every one of these is stopped at a gate and needs YOUR tap on a Telegram "Approve" button before it runs
--	---

Everything else in the architecture exists to serve that split safely and cheaply.

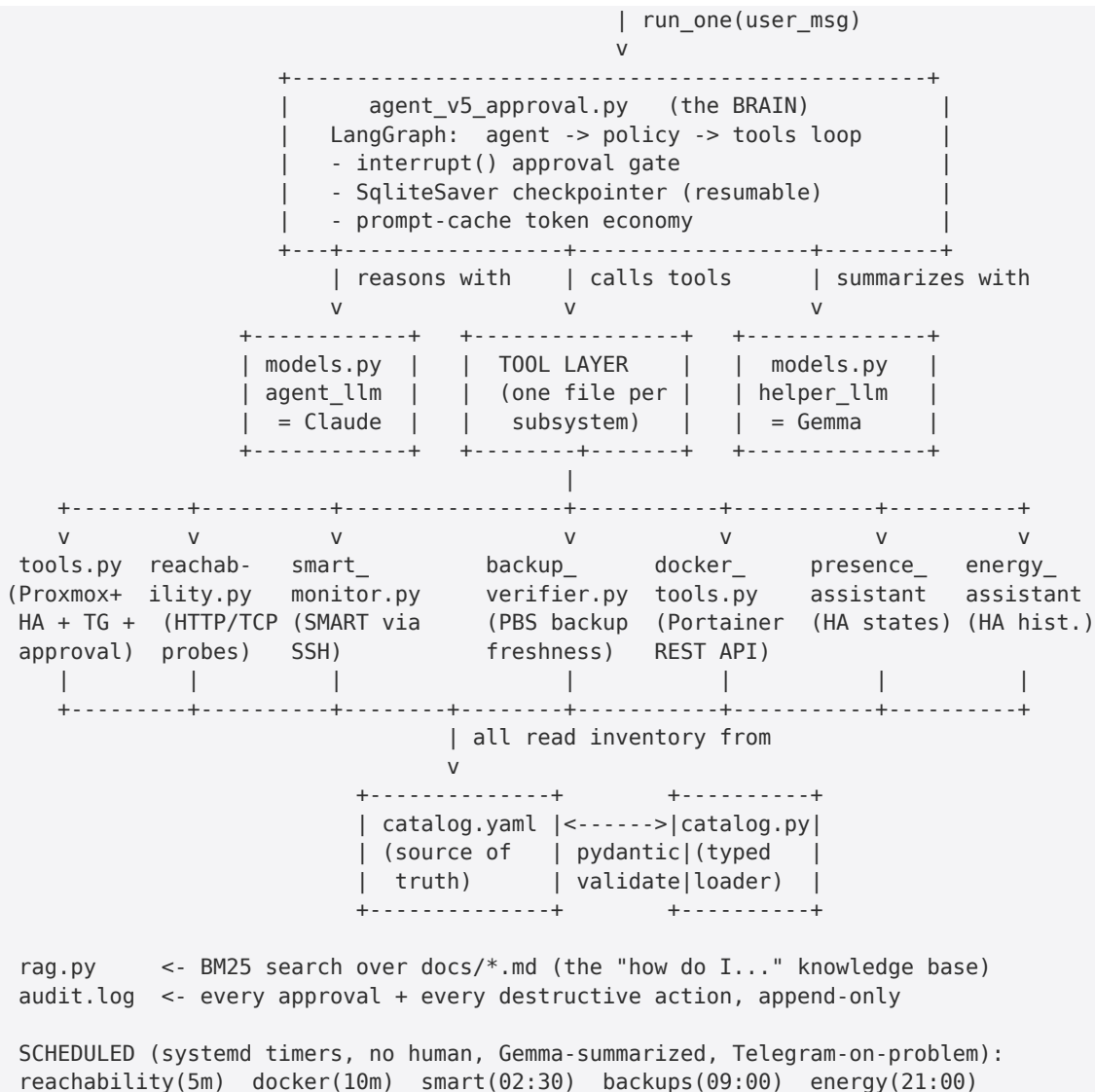
A second big idea: two brains.

- Claude (cloud) does the reasoning — multi-step tool-calling loops. Reliable, but costs money per run.
- Gemma (local, on your own LXC) does the summarizing — "turn this JSON into one sentence." Free, runs offline, used by every scheduled monitor.

This is why the monitors keep working even when the Anthropic balance hits zero.

Part 1 — The whole system on one page

YOU ---Telegram---> sentinel_bot.py (long-running, single poller)	+-----+-----+-----+
---Alexa-----> voice_server.py --> voice.py	
	+-----+-----+-----+



Keep this picture in mind. Now we build up to the brain.

Part 2 — The agent evolution (the actual course)

The five `agent_v*.py` files solve the same task ("check these containers, fix what's broken") five times, each adding one production concern.

Lesson 2.1 — `agent\_v1\_raw.py`: the ReAct loop with zero frameworks

This is the most important file to understand, because every framework is just sugar over this loop. An "agent" is fundamentally a `while` loop around an LLM that can call functions.

The tool definitions are hand-written JSON Schemas — the raw format the Anthropic API actually wants:

```

TOOLS = [
    {
        "name": "check_proxmox_status",
        "description": "Get status (running/stopped, memory %, uptime)...",
        "input_schema": {
            # JSON Schema. Claude reads this to
            "type": "object",
            # know what arguments to produce.
            "properties": {
                "node": {"type": "string"},
            }
        }
    }
]

```

```

        "vmid": {"type": "integer"},
    },
    "required": ["node", "vmid"],
}, ...
]

```

TOOL_FUNCS is the other half — a name -> function dispatch table. The LLM names a tool; this dict turns the name back into a real Python function.

The loop itself, annotated line by line:

```

def run_agent(user_msg, max_iters=10):
    messages = [{"role": "user", "content": user_msg}] # conversation history

    for i in range(max_iters):
        # max_iters is a SAFETY rail: it
        # stops an agent that loops forever.

        resp = client.messages.create(
            model="claude-sonnet-4-6",
            system=SYSTEM, # the agent's job description + rules
            tools=TOOLS,   # what it is allowed to call
            messages=messages, # everything so far
        )

        messages.append({"role": "assistant", "content": resp.content})
        # ^ ALWAYS append the model's reply first, or the next turn loses context.

        if resp.stop_reason == "end_turn": # Claude decided it is DONE (no tool).
            return # print final text and stop.

        # Otherwise stop_reason == "tool_use": Claude wants to call something.
        tool_results = []
        for block in resp.content: # one reply can request MANY tools
            if block.type == "tool_use":
                result = TOOL_FUNCS[block.name](**block.input) # actually run it
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id, # MUST echo the id back so
                    "content": json.dumps(result), # Claude pairs result<->request
                })

        messages.append({"role": "user", "content": tool_results})
        # ^ tool results go back in as a "user" turn. Loop again; now Claude can
        # reason about what it learned.

```

That is the ReAct pattern (Reason -> Act -> Observe -> repeat):

```

user_msg
|
v
+-----+ stop_reason == "tool_use" +-----+
| Claude | -----> | run tools, |
|         | <----- | feed results |
+-----+ results appended as msgs +-----+
| stop_reason == "end_turn"
v
final answer

```

Three protection mechanisms already live here: **max_iters** (loop bound), the **tool_use_id** echo (correctness), and **system** rules like "Always check status before restarting" (behavioral guardrail in natural language).

Lesson 2.2 — `agent\_v2\_langchain.py`: let the framework write the loop

v2 does the exact same thing with about a third of the code. Two changes worth learning.

1. The `@tool` decorator replaces hand-written JSON Schema:

```
@tool
def check_proxmox_status(node: str, vmid: int) -> dict:
    """Get status (running/stopped, memory %, uptime) of a Proxmox LXC or VM.
    Args:
        node: Proxmox node name, e.g. 'pve'
        vmid: VM/LXC ID, e.g. 100
    """
    return _check_proxmox_status(node, vmid)
```

LangChain reads the function signature (`node: str, vmid: int`) and the docstring and generates the JSON Schema you wrote by hand in v1. The docstring is not a comment — it is the prompt the LLM reads to decide when to use the tool. Notice the pattern: the decorated function just forwards to `_check_proxmox_status` imported from `tools.py`. The real logic lives elsewhere; this is a thin "expose to the LLM" wrapper. That pattern repeats all the way to v5.

2. The whole loop becomes one call:

```
agent = create_agent(model=llm, tools=tools, system_prompt=SYSTEM)
result = agent.invoke({"messages": [{"role": "user", "content": user_msg}]})
```

`create_agent` builds and runs the same Reason -> Act -> Observe loop internally. Lesson: the framework did not add magic — it hid the `while` loop you now understand.

Lesson 2.3 — `agent\_v3\_langgraph.py`: make the loop a graph you can edit

v2's one-liner is a black box. v3 rebuilds it as an explicit graph so you can insert new steps (which is exactly what the approval gate will be). Three LangGraph concepts you must own:

STATE — what flows through the graph:

```
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
```

`add_messages` is a reducer: when any node returns `{"messages": [x]}`, `x` is appended to the list, not overwritten. It is the graph-native version of v1's `messages.append(...)`.

NODES — functions that take state and return a state update:

```
def agent_node(state):
    # the "Reason" step
    response = llm_with_tools.invoke(
        [SystemMessage(content=SYSTEM)] + state["messages"])
    return {"messages": [response]}

tool_node = ToolNode(tools)
# the "Act" step (prebuilt: reads the last
# message's tool_calls, runs them, returns
# the results)
```

EDGES — the wiring:

```
graph.add_edge(START, "agent")
graph.add_conditional_edges("agent", tools_condition)
graph.add_edge("tools", "agent")
app = graph.compile()
```

`tools_condition` is the router: if the last AI message contains `tool_calls`, go to "tools", else END. That IS v1's `if resp.stop_reason == "end_turn"` check, expressed as a graph edge:

```

+-----+
START -->| agent |<-----+
      +---+---+
      |tools_condition|
      |             |
has tool_calls? no   |
      v             v
+-----+          END
```

```
| tools |-----+
+-----+
```

Same loop as v1. But now it is data you can rewire — and rewiring it is the whole point of v5.

Note: `agent_v3_langgraph.py` has a typo on its import line (`from tools import (LXC)` — a stray token that would crash on import. v3 is a teaching artifact, not run in production (the bot uses v5), so it never bit anyone, but it is worth knowing it is there.

Lesson 2.4 — `agent\_v4\_langsmith.py`: observability

v4 is v3 plus tracing. The lesson: you cannot operate an agent you cannot see. A multi-step tool loop fails in non-obvious ways (wrong tool, bad argument, infinite retry), and you need a timeline.

The entire activation is environment variables:

```
os.environ.setdefault("LANGSMITH_TRACING", "true")      # master switch
os.environ.setdefault("LANGSMITH_PROJECT", "homelab-sentinel")
```

Once those are set, every LLM call, tool call, and graph step auto-uploads to the LangSmith UI — zero code changes. The `@traceable` decorator instruments your own non-tool helpers so they appear as spans too, and a `config` block with `tags/metadata/run_name` makes runs searchable.

In production, Sentinel swapped LangSmith for a cheaper homemade alternative: the `_log_usage` + `audit.log` mechanism in v5. Same goal, no external dependency.

Lesson 2.5 — `agent\_v5\_approval.py`: the production brain

v5 = v3's graph plus three new production concerns.

(a) The graph gains a `policy` node — the human-in-the-loop gate

In v3 the wiring was `agent -> tools`. v5 inserts a checkpoint between them:

```
graph.add_edge(START, "agent")
graph.add_conditional_edges("agent", tools_condition,
                             {"tools": "policy", END: END})
# ^ KEY CHANGE: when the agent wants tools, we do not go straight to "tools" -
# we route to "policy" FIRST.
graph.add_edge("policy", "tools")
graph.add_edge("tools", "agent")
```

```

+-----+
START -->| agent |<-----+
+-----+
tools_condition
|
wants tools?  done
  v           v
+-----+    END
| policy | <- NEW: checks if any
+-----+    requested tool is
|         |    destructive
  v
+-----+
| tools |-----+
+-----+ (gated execution)
```

The `policy_node` is the heart of the safety model:

```
def policy_node(state):
    last = state["messages"][-1]
    tool_calls = getattr(last, "tool_calls", None) or []
    destructive = [tc for tc in tool_calls if tc["name"] in DESTRUCTIVE_TOOLS]

    if not destructive:
```

```

        return {"denied_ids": []}          # all-safe batch -> sail through, no prompt

    decisions = interrupt({                # THE MAGIC. Pause the entire graph.
        "kind": "approval_request",
        "destructive_calls": [
            {"tool_call_id": tc["id"], "name": tc["name"], "args": tc["args"]}
            for tc in destructive
        ],
    })

    denied_ids = [d["tool_call_id"] for d in (decisions or [])
                  if d.get("decision") != "approved"]
    return {"denied_ids": denied_ids}

```

`interrupt()` is LangGraph's pause button. When called, the graph stops, saves its entire state to disk, and returns control to the caller with the payload you passed. The caller (your phone, via Telegram) decides, then resumes the graph by handing back the decisions. Note the fail-safe default: anything not explicitly "approved" (denied, timeout, error) lands in `denied_ids`. Silence = no.

`DESTRUCTIVE_TOOLS` is the policy itself, as plain data — three names:

```
DESTRUCTIVE_TOOLS = {"restart_lxc", "restart_docker_container", "call_ha_service"}
```

Want to gate a new tool? Add its name to that set. That is the entire policy surface.

(b) The ``gated_tool_node`` — enforce the decision

v3 used the prebuilt `ToolNode`, which blindly runs every requested tool. v5 cannot use it, because some calls in the batch may have been denied. So v5 hand-writes the executor:

```

def gated_tool_node(state):
    last = state["messages"][-1]
    tool_calls = getattr(last, "tool_calls", None) or []
    denied_ids = set(state.get("denied_ids", []) or [])

    out_messages = []
    for tc in tool_calls:
        if tc["id"] in denied_ids:
            out_messages.append(ToolMessage(
                content="REFUSED by operator. Do not retry without new "
                "investigation.",
                tool_call_id=tc["id"], name=tc["name"]))
            continue # skip execution entirely
        tool_fn = _TOOLS_BY_NAME.get(tc["name"])
        try:
            result = tool_fn.invoke(tc["args"]) # approved -> actually run
        except Exception as e:
            result = {"error": f"{type(e).__name__}: {e}"} # tools never crash graph
        out_messages.append(ToolMessage(content=str(result),
                                         tool_call_id=tc["id"], name=tc["name"]))

    return {"messages": out_messages, "denied_ids": []} # clear for next loop

```

The elegant part: a denial is not an exception — it is a `ToolMessage` fed back to Claude saying "REFUSED." On its next turn Claude sees the refusal and can reason ("the operator declined the restart; I will just report the problem instead") rather than blindly retrying. The system prompt reinforces this: "if denied, you will see a refusal `ToolMessage` — reason about it and respond, do not blindly retry."

(c) The checkpointer — survive a restart mid-approval

v5 wires in `SqliteSaver`. When the graph hits `interrupt()`, its full state is persisted to `checkpoints`. `sqlite` keyed by a `thread_id`. Consequences:

- You can be at dinner for ten minutes before tapping Approve; the agent is not

"running" in memory — it is a saved row.

- The process can crash and restart; resuming the `thread_id` continues exactly where it paused.
- Each Telegram chat gets its own `thread_id` (`chat-<id>`), so conversations have independent memory.

(d) The caller loop — where interrupt/resume actually happens

```
def _execute(user_msg, thread_id, approval_fn, checkpointer, verbose, reuse=False):
    app = ... # compiled graph (cached for bot)
    config = {"configurable": {"thread_id": thread_id}} # which conversation
    state = {"messages": [{"role": "user", "content": user_msg}], "denied_ids": []}

    result = app.invoke(state, config=config) # run until END or interrupt

    while result.get("__interrupt__"): # graph paused for approval
        payload = result["__interrupt__"][0].value
        decisions = []
        for call in payload["destructive_calls"]:
            d = approval_fn( # ask the human (Telegram)
                action=f"{call['name']}({pretty_args})",
                details=f"The agent wants to run: {call['name']}(...)"
            )
            decisions.append({"tool_call_id": call["tool_call_id"],
                            "decision": d["decision"], "by": d.get("by")})
        result = app.invoke(Command(resume=decisions), config=config) # RESUME
    return result["messages"][-1].content # final answer
```

The `while` is what lets one user message survive several approval rounds (the agent might restart container A, get approval, then propose restarting container B). `Command(resume=decisions)` is the literal "un-pause" call, and `decisions` becomes the return value of `interrupt()` back inside `policy_node`. The loop closes:

```
_execute --app.invoke--> graph runs --interrupt()--> returns __interrupt__
      ^                                     |
      |                                     v
      +-- app.invoke(Command(resume=decisions)) <-- approval_fn (Telegram tap)
```

Notice `approval_fn` is injected, not hard-coded. The CLI passes `request_telegram_approval` (its own poller); the bot passes `request_approval_via_bot` (event-based); the voice path passes `_voice_deny` (auto-deny everything -> read-only). Same graph, three different "ask the human" strategies. This dependency-injection is what makes one brain serve three front-ends safely.

(e) The token economy (the cost constraint, in code)

Because Claude costs money and the account ran dry, v5 is laced with token optimizations. Three to learn.

Prompt caching — the system prompt + tool schemas are byte-identical on every call, so mark them cacheable:

```
_SYSTEM_MSG = SystemMessage(content=[
    {"type": "text", "text": SYSTEM, "cache_control": {"type": "ephemeral"}}
])
```

During one investigation the agent loops 5-10 times; each loop re-sends that big prefix. With the cache breakpoint, re-reads cost about 10% of normal input tokens. `_cache_tail` adds a second breakpoint on the conversation tail so prior messages are cached too.

History bounding — `_bounded_history` caps replayed messages at `MAX_HISTORY_MESSAGES` (40) using `trim_messages(..., start_on="human")`. The `start_on="human"` matters: it never cuts in the middle of a `tool_call` / `tool_result` pair (an orphaned tool result is an API error). Without this, a long-lived Telegram chat's per-message cost grows unbounded.

Usage auditing — `_log_usage` writes token counts (including cache hits) to `audit.log`, so you can watch the savings live: `tail -f audit.log | jq 'select(.event=="llm_usage")'`. This is the homemade replacement for LangSmith from v4.

Part 3 — The foundation layer

The agent is only as good as the three modules under it.

`models.py` — the two-brain factory

This module exists so that no other file ever instantiates an LLM directly. Two factories:

- `agent_llm()` -> `ChatAnthropic` (Claude). For anything that calls `.bind_tools()` or loops.
- `helper_llm()` -> `ChatOpenAI` pointed at your local llama.cpp server (`http://192.168.178.75:8383/v1`). For single-shot summarize / classify.

The docstring states the why: Gemma-4B cannot reliably call structured tools across multi-turn loops (it hallucinates tool names or breaks JSON), but it is excellent and free at "summarize this in one sentence." The design principle: route each task to the cheapest model that can do it reliably. The payoff: the day you give the LXC more RAM and want a local model to run the agent too, you change one line here and the whole system follows.

One subtle line: `extra_body={"chat_template_kwargs": {"enable_thinking": False}}` — the Gemma server runs with a reasoning budget, so by default it would "think" for the whole completion and return empty content. This disables thinking for single-shot helper tasks. (It is also why several monitors lift temperature to 0.2 — at temp 0 Gemma sometimes emits empty completions for "everything's fine" prompts.)

`catalog.yaml` + `catalog.py` — the source of truth

The agent should never guess what is in your homelab. `catalog.yaml` declares it: every VM/LXC with its `vmid`, `criticality`, `restart_policy`, backup expectations, and network `endpoints`. This is operator policy as data. The two fields that drive all safety behavior:

```
- vmid: 105
  name: OPNSense
  criticality: critical          # how loud to alert
  restart_policy: never        # agent is FORBIDDEN to restart the router
```

`catalog.py` loads and validates it with pydantic models. The reason is the best one-liner in the codebase: "Typos in YAML fail fast at load time, not three days later in a monitor at 3am." If you write `criticality: critcal`, the `Literal["critical", "high", "medium", "lab"]` type rejects it on load with a pointer to the bad field.

The class gives the agent clean lookups — `get(name)`, `by_vmid(vmid)`, `filter(criticality=...)` — and `@lru_cache` means the file is parsed once per process, not per tool call. `_merge_defaults` lets a service omit common fields and inherit them from the top-level `defaults:` block.

`tools.py` — where the agent touches the real world

This is the biggest module (about 855 lines) and the integration layer. It speaks raw HTTP/SSH to Proxmox, Home Assistant, and Telegram. Five things to learn from it.

1. Every tool returns a dict, never raises. A 401 becomes `{"error": "auth`

failed..."}; a connection failure becomes {"error": "cannot reach Proxmox..."}`. The agent's loop feeds these dicts straight back to Claude, who reads the error text and reasons about it. An exception would crash the graph; a dict is just another observation. Core agentic-design rule: tools degrade, they do not explode.

- 2. The audit log. `_audit(event, payload)` appends one JSON line to `audit.log` for every approval and every destructive action. It is wrapped in a bare `try/except: pass` — "auditing must never break the agent." The log is your forensic trail.
- 3. The QEMU memory trap — genuine homelab knowledge encoded in code. `get_guest_mem_pct` exists because Proxmox's host-side memory reading for a QEMU VM is a lie — it counts the Linux page cache, so a perfectly healthy VM shows 85%+ "used." Restarting on that number would reboot healthy machines. So for QEMU, Sentinel SSHes in and reads the real number from `/proc/meminfo (1 - MemAvailable/MemTotal)`. The status dict carries a `mem_pct_source` field (`guest / host_cgroup / host_balloon`) so Claude knows whether to trust it, and the system prompt has explicit rules: "'host_balloon' -> UNRELIABLE... call `get_guest_mem_pct` before any restart decision." This is the kind of domain protection that separates a toy from a tool.
- 4. The approval gate primitive. `request_telegram_approval` is the standalone (CLI / monitor) version of the gate. Its safety design:
 - A random `token` correlates this request to its button press, so two simultaneous approvals cannot be confused.
 - It snapshots the latest Telegram `update_id` before sending so it ignores stale button clicks from old requests.
 - It long-polls up to `timeout` seconds; no click -> `decision = "timeout"` -> treated as deny.
 - It edits the original message to "APPROVED by @user (4.2s)" so the chat history itself is a second audit trail.
- 5. The `*_raw` vs gated twin pattern. Every destructive tool has two forms:

```
def restart_lxc_raw(node, vmid):    # NO approval. Caller must gate. Audited.
def restart_lxc(node, vmid):       # asks request_telegram_approval, THEN calls _raw
```

Why two? Because v5 gates at the graph level (`policy_node`), so it must import the `_raw` variants — otherwise you would get double prompting (graph asks, then the tool asks again). Simpler agents (v3 / v4) that have no graph gate use the self-gating `restart_lxc`. The rule: pick ONE variant per agent — never mix them on the same call.

Part 4 — Protection mechanisms, all in one place

Here is the complete defense-in-depth, layer by layer. A destructive action has to survive all of these to happen:

Layer 0	catalog policy	restart_policy:"never" -> tool is never proposed
	-----	(OPNSense router, Windows VM, pentest VM)
Layer 1	system-prompt rules	"check status before restarting"; presence-aware
	-----	HVAC; distrust host_balloon memory; no retry on
		refusals
Layer 2	read-before-write	agent must call check_* tools before a destructive
	-----	one (enforced by prompt + real-state tool results)
Layer 3	policy_node gate	interrupt() pauses the graph for ANY destructive
	-----	call (the hard, code-level stop)
Layer 4	human approval	you physically tap Approve/Deny on Telegram, per
	-----	action. default-deny: timeout/error/denied = no
Layer 5	gated_tool_node	denied id -> synthetic "REFUSED" msg, tool never
	-----	runs

Layer 6	audit.log + chat edit	every ask + every execution logged two ways, append-only
Layer 7	auth boundaries	bot answers only AUTHORIZED_CHAT_IDS; voice is read-only; Proxmox API token (scoped); Portainer token (revocable); voice bearer token; SSH BatchMode (fail fast)
Layer 8	blast-radius limits	max_iters loop bound; tools return dicts not exceptions; max_tokens caps; AdGuard-DNS restart warning in prompt

The clearest single example: ask Sentinel "restart the router." Layer 0 (catalog says **never**) plus Layer 1 (prompt rule: "If restart_policy='never', DO NOT call restart_lxc — alert instead") mean it will not even try. If somehow it did, Layer 3 would still pause and Layer 4 would still require your tap. No single failure lets it through.

The read-only-by-construction voice path is a beautiful instance:

```
def _voice_deny(action, details, timeout_s=None):
    return {"decision": "denied", "by": "voice (read-only)"}
```

By injecting an approval function that always denies, the exact same agent becomes physically incapable of destructive action over voice — no separate "read-only mode" code needed. The gate architecture gives you that for free.

Part 5 — The monitors (learn one, you know five)

`reachability.py`, `smart_monitor.py`, `backup_verifier.py`, `docker_tools.py`, `energy_assistant.py`, and `presence_assistant.py` all follow one shape:

+-----+	+-----+	+-----+	+-----+
read catalog -->	probe/query the -->	classify into a -->	summarize_*()
for targets	real world (//)	structured digest	on local Gemma
+-----+	+-----+	+-----+	+-----+
Each module exposes the SAME function 3 ways:			v
1. as an @tool in agent_v5 (Claude calls it)			deterministic fallback
2. as a CLI (python reachability.py --alert)			if Gemma is down – the
3. as a systemd timer job (scheduled, headless)			monitor NEVER goes
			silent

Take `reachability.py` as the archetype:

- `probe_endpoint` probes ONE endpoint (TCP connect or HTTP GET), returns a structured dict with status up / down / wrong_code / auth_missing. Never raises.
- `sweep_services` runs all probes in parallel via `ThreadPoolExecutor` (16 workers), so checking nine services takes as long as the slowest one, not the sum. Rolls up counts plus a `critical_down` list.
- `summarize_sweep` hands the compact result to Gemma for a 2-3 sentence digest, with a deterministic `_fallback` if Gemma is unreachable. This is the resilience pattern: the LLM makes the message nicer, but its absence never stops the alert.
- `main()` is the CLI. `--alert` sends Telegram only if something critical is down. Exit code 2 on critical-down so systemd / cron can react.

The others just swap the middle:

- `smart_monitor.py` — SSHes to Proxmox, runs `smartctl -aj`, parses SATA vs NVMe attributes (reallocated sectors, NVMe wear / spare) against thresholds. Subtlety: `smartctl`'s exit code is a health bitfield, not a success flag, so it parses stdout JSON first and only treats the SSH error as fatal if there is no JSON at all.

- `backup_verifier.py` — queries Proxmox storage for the newest backup per vmid, compares age to each service's `max_backup_age_h` -> fresh / stale / missing / skipped.
- `docker_tools.py` — talks to the Portainer REST API (not SSH) with an X-API-Key token; `restart_container_raw` is the gated-at-graph-level destructive tool. This is why restarting AdGuard carries a DNS-outage warning in the system prompt.
- `presence_assistant.py` — pure read-only HA state readers (`check_presence_state`, `check_light_state`, `check_climate_state`). Its docstring states the philosophy: "This module deliberately has NO rule engine — the LLM plus the catalog policy ARE the rules."
- `energy_assistant.py` — the trickiest: `_delta_for_entity` implements a reset-aware counter delta because Tuya plugs reset their lifetime totals about daily; a naive last-minus-first would straddle resets and under-report, so it sums only the upward steps.

Part 6 — ``rag.py``: the "how do I..." knowledge base

When you ask a procedural question ("how do I restart the bot?", "which storage holds backups?"), live-state tools cannot answer — that knowledge lives in your markdown notes in `docs/`. `rag.py` is Retrieval-Augmented Generation over those docs, and it is deliberately BM25 lexical, not embeddings. The two reasons:

1. The local llama-server serves a chat model only; switching it to embeddings would knock out the chat model the agent + monitors depend on.
2. Python 3.14 has no torch / onnxruntime wheels yet, so a semantic embedder means compiling from source on a RAM-limited LXC.

BM25 genuinely fits homelab docs — queries share exact vocabulary (service names, IPs, config keys) with the text. The flow:

```
docs/*.md --ingest()--> chunk by heading --> rag_index.json (human-readable)
                                     |
query --_tokenize--> BM25 score each chunk <-----+
                                     |
                               +-----+-----+
                               |               |
search(): top-k chunks         answer(): chunks + local Gemma
(used by the agent's          (used by CLI; ZERO Claude tokens)
search_docs tool)
```

- `_tokenize` keeps internal dots so `192.168.178.9` and `sensor.refrigerator_total_energy` survive as single tokens.
- `_chunk_text` groups paragraphs under their nearest markdown heading — the heading becomes a citation breadcrumb.
- `_BM25` is the textbook Okapi BM25 formula (term frequency times inverse-document-frequency, length-normalized), about 50 lines of pure Python, no dependencies.
- `search()` is exposed to the agent as `search_docs`; `answer()` does retrieval + Gemma generation for the CLI — both cost zero Claude tokens.

The clean interface (just `search()`) is the lesson: when Python catches up on wheels, you swap BM25 for embeddings behind that one function and no caller changes.

Part 7 — ``sentinel_bot.py``: the concurrency story

This is the long-running process (a systemd service) that turns Telegram into a chat with the agent. The headline design problem and its solution:

The single-poller rule. Telegram delivers updates via `getUpdates`, and only one poller can consume them (a second poller steals the other's messages). The bot is THE poller. But the agent also needs to ask for approvals, which normally means polling Telegram too. Conflict. The fix — dependency injection again:

- The bot replaces the agent's approval function with `request_approval_via_bot`.
- That function sends the approval message, then blocks on a `threading.Event` instead of polling.
- The bot's single poller receives the button click as a `callback_query`, looks up the pending Event by token, and `.set()`s it, unblocking the waiting agent thread.



Other production details worth seeing:

- Per-chat serialization — each chat gets a `threading.Lock`, so two rapid messages in one chat run one at a time (the agent's per-chat memory cannot be corrupted by concurrency), while different chats run in parallel.
- Authorization — messages from any `chat_id` not in `AUTHORIZED_CHAT_IDS` are silently dropped. This is the bot's front door lock.
- The typing indicator — a background thread re-sends "typing..." every 4s while the agent (which can take 20s+) works, so the chat does not look dead.
- Message splitting — Telegram caps messages at 4096 chars; `_send_message` splits at newlines.
- `/reset` — switches the chat to a fresh `thread_id`, abandoning (not deleting) old checkpoint state — a new conversation memory.
- Startup offset skip — on boot it skips already-queued updates so it does not reply to yesterday's messages.

Part 8 — `voice.py` / `voice\_server.py`: hands-free, zero-token

This path is Alexa-native — Amazon does the speech-to-text, there is no Whisper. The chain:

```
"Alexa, <phrase>" -> Alexa Routine -> HA automation
-> POST http://192.168.178.106:8099/voice {"intent":"status"}
-> voice_server.py -> voice.handle_intent() -> answer string
-> speak_on_alex() -> Echo speaks it (Alexa Media Player TTS)
```

- `voice_server.py` is a tiny FastAPI app: `/voice` runs an intent, `/speak` says

arbitrary text, `/health` lists intents. Bearer-token auth, with a loud warning if the token is unset.

- `voice.py`'s `INTENTS` dict maps words -> handlers. The common intents (status, backups, energy, disks, presence, docker) just call the existing monitors and summarize on Gemma -> zero Claude tokens, so voice works with an empty Anthropic balance.
- Only the free-form `ask` intent uses Claude — and it is forced read-only via `_voice_deny`, plus it catches the "out of credits" error and degrades gracefully to "I can only run the built-in checks right now."

This is the cost constraint expressed as architecture: the things you will say 95% of the time are free and offline; only open-ended reasoning touches the paid model.

Part 9 — Deployment: systemd, the LXC, the cost design

`/opt/sentinel` IS the live LXC (Proxmox LXC 106) — flat layout, no git, edited in place. `systemd` runs everything:

- One long-running service — `sentinel-bot.service: Type=simple, Restart=on-failure, KillSignal=SIGINT` (clean shutdown). Plus `sentinel-voice.service` for the FastAPI bridge.
- Five oneshot + timer pairs — the monitors. e.g. `sentinel-reachability.timer` fires `OnUnitActiveSec=5min` and runs `reachability.py --alert --no-summary` as a `Type=oneshot, --no-summary` in the timer = skip even the Gemma call for the routine sweep; alert only fires on real problems.

```
systemd
|-- sentinel-bot.service      (always on)  -- Telegram chat
|-- sentinel-voice.service    (always on)  -- Alexa bridge :8099
+-- timers --+
| - reachability              every 5 min  -- alert if critical down
|   |-- docker                every 10 min -- alert if container down
|   |-- smart                 02:30 nightly -- alert if disk failing
|   |-- backups               09:00 daily  -- alert if backup stale
|   +-- energy                21:00 daily  -- always sends digest
```

The cost design across the whole system:

WHO RUNS	WHAT MODEL	WHEN	COST
-----	-----	----	----
5 monitors	Gemma (local)	scheduled, always	\$0
voice common	Gemma (local)	on demand	\$0
RAG answer()	Gemma (local)	CLI	\$0
the agent (bot)	Claude	only when you chat	~\$0.005/run, cached
voice "ask"	Claude	rare free-form Q	paid, degrades gracefully

How to study this yourself (learn-by-doing)

```
cd /opt/sentinel

# 1. SEE the agent loop with your own eyes (the whole course in one run):
uv run python agent_v1_raw.py          # add print(resp.stop_reason) to watch it

# 2. Compare the SAME task across abstraction levels:
#   diff the structure of agent_v1 -> v2 -> v3 side by side in your editor.

# 3. Watch the production agent reason + gate, live:
```

```
uv run python agent_v5_approval.py # then tap Approve/Deny on Telegram
```

```
# 4. Watch tokens + approvals stream as it runs:  
tail -f audit.log | jq .
```

```
# 5. Run a monitor by hand and read its structured output:  
uv run python reachability.py --json | jq .  
uv run python smart_monitor.py  
uv run python docker_tools.py
```

```
# 6. Query your docs offline (zero Claude tokens):  
uv run python rag.py "how do I restart the bot?"
```

Read the files in this order, and stop at each to predict what the next adds: agent_v1 -> v2 -> v3 -> v5 (skip v4 unless you want LangSmith), then tools.py (the world), then catalog.py + catalog.yaml (the policy), then sentinel_bot.py (the concurrency), then any one monitor.

The one sentence to remember

An agent is a **while** loop over an LLM with tools (v1); a framework just makes that loop into editable data (v3); and "editable" is what lets you insert a human-approval gate that physically stops every dangerous action (v5) — with a cheap local model and prompt caching keeping it nearly free to run.